A CASE-STUDY BASED REPORT

ON

CineMatch - Movie Recommendation Engine

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Mydam Aswitha Patel **2520030001**

Panapakam Hasini **2520030282**

Under the Esteemed guidance of

Dr.G.S.Raghavendra

Asst.Professor, Dept. of CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction.....

Background.....
Motivation for the Project.....
Overview of the Work Carried Out .....
Scope and Responsible-Use Statement.....

Problem Statement.....

Case-Study Scenario.....
Central Case-Study Question.....
Formal Problem Definition.....
Objectives that Guide the Study .....
Assumptions and Limitations .....

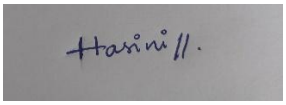
Dataset and Exploratory Data Analysis

About the Dataset .....
Purpose of EDA.....
Preparation Steps Actually Performed.....
EDA Exploration - Implementation.....
Summary of EDA Findings .....

Algorithm Selection and Implementation.....

Conclusion





Signature of the Team members

Abstract

CineMatch is a Streamlit web application that implements two separate movie recommendation pipelines, content-based filtering and collaborative filtering, and compares their output directly. Content-based filtering uses TF-IDF and cosine similarity over movie genres, decade and title words. Collaborative filtering uses SVD matrix factorization trained on user ratings. Both are built from three MovieLens-style CSV files (movies, ratings, links).

The central finding is that the two methods recommend almost entirely different movies: their top-10 lists overlap by about 0.2%. SVD predicts ratings slightly better (RMSE about 0.898) than content-based scoring (about 0.909) and an item-mean baseline (about 0.961). Content-based filtering is the better fit for "movies like this one", because the ratings data is sparse (about 4.9 ratings per user).

Around the comparison, the application provides a cinematic OTT-style interface with login, watch-history personalization, mood-based picks, discovery features and TMDB posters.

Introduction

Background

Streaming platforms hold catalogues far too large to browse, so they rely on recommender systems to suggest what to watch next. Two classic families dominate. Content-based filtering recommends items whose descriptions resemble something the user already liked. Collaborative filtering recommends items that users with similar rating behaviour liked, without looking at what the items are.

Motivation for the Project

The two families fail and succeed in different situations. Content-based filtering works for any movie with metadata, even one nobody has rated, but it cannot see taste beyond genres and titles. Collaborative filtering can find links that metadata hides, but it depends on enough ratings. A project that shows only one method cannot show this difference, so CineMatch implements both and compares their output on the same movie or user. A second aim is to present the result as a believable movie-discovery product, not a data notebook.

Overview of the Work Carried Out

- Inspected the three CSV files and cleaned, validated and joined them on `movieId`.
- Built a content-based pipeline with TF-IDF and cosine similarity.
- Built a collaborative pipeline with SVD matrix factorization and measured its RMSE.

- Built a comparison module that shows overlap, rank shifts and scores for both methods, plus a leak-free offline evaluation.
- Built a Streamlit application around the comparison: login, watch history, mood and discovery features, My List, taste profile and achievements.
- Added a TMDB poster system with scene-image and generated-card fallbacks, and an age-rating filter on artwork.
- Wrote engine tests and scripted interface scenarios.

Scope and Responsible-Use Statement

Scope: -

CineMatch is an offline prototype built on MovieLens-style data. It has a local login, no email verification, and no study with real viewers. Its accuracy numbers describe this dataset only.

Responsible use: -

- No feature claims to understand emotions or personality. A mood is a viewing mood the user selects, mapped to genres. Discovery mode is a diversity control. The movie profile label comes only from the genre counts in the watch history.
- Every explanation shown to the user is supported by the algorithm that produced the result.
- Artwork is filtered for age rating (strict by default). The filter applies to posters and scene images only. It does not remove adult-rated titles from recommendation lists, and TMDB cannot flag an individual suggestive poster on a film with a family rating.
- The TMDB API key is read from Streamlit secrets or the `TMDB_API_KEY` environment variable, not from source code. The secrets file must not be shared publicly, and the key should be rotated after the demo.
- TMDB attribution is shown in the footer, as its terms require.

Problem Statement

Case-Study Scenario

A small streaming service has a catalogue of about 4,464 movies and a MovieLens-style ratings file in which each user has rated only about 4.9 movies on average. A viewer who has just watched a movie such as Toy Story wants a suggestion for what to watch next. The service can recommend movies that look similar, or movies that similar viewers liked. The case study builds both and examines what each produces.

Central Case-Study Question

How does content-based filtering differ from collaborative filtering, and which recommendations does each method produce for the same movie or user?

Formal Problem Definition

Let U be the set of users, M the set of movies, R the observed ratings r(u, i), and X(i) the metadata of movie i (genres, decade, title words). The task is to build two separate functions and compare them:

- a content-based ranking of movies for a selected movie i, using only X;
- a collaborative ranking of movies for a user u, using only R.

The comparison reports the overlap of their top-10 lists, the rank of each movie in each list, and two accuracy measures. The first is rating prediction error on held-out ratings T:

$$\text{RMSE} = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{ui} - \hat{r}_{ui})^2}$$

The second is hit rate at 10: the share of users whose hidden liked movie appears in their top 10.

Objectives that Guide the Study

1. Implement a genuine content-based pipeline using TF-IDF and cosine similarity.
2. Implement a genuine collaborative pipeline using SVD matrix factorization on user ratings.
3. Keep the two pipelines logically separate, so neither reuses the other's signal.
4. Compare them directly: overlap, ranking differences, scores and a leak-free offline evaluation.
5. Wrap the comparison in a usable product: search, watch history, mood and discovery features, login and real movie posters.

Assumptions and Limitations

Assumptions

- A rating of 4 or higher means the user liked the movie. This is used to pick the hidden movie in the evaluation.
- Genres, decade and title words are a usable description of a movie's content.
- A new visitor with no user id can be matched through the movies they mark as watched.

Limitations

- The CSVs have no keywords, cast or plot, so content features are limited to genres, decade and title words.
- Ratings have no timestamps, so nothing time-based is claimed. There is no language or country column, so the world-discovery feature works only with TMDB metadata.
- Ratings are sparse (about 4.9 per user), so SVD factors are noisy and all hit rates are small.
- The evaluation is offline. No real viewers were tested.
- During development Surprise could not be installed, so tests ran on the NumPy fallback. The interface was tested with a stubbed Streamlit and a simulated TMDB, and the layout was not seen in a real browser before this document was written.
- TMDB is blocked on some networks. When it cannot be reached, every card shows generated artwork. The strict age filter also shows generated cards for any movie TMDB does not confirm as G, PG or PG-13 style.
- The login is local, with hashed passwords, and has no email verification or password reset.

Dataset and Exploratory Data Analysis

About the Dataset

The application loads three MovieLens-style CSV files from the `data/` folder and joins them on `movieId`, never on title.

File	Contents	Key columns
movies	Movie catalogue	movieId, title, genres, tmdbId
ratings	User ratings	userId, movieId, rating
links	External IDs	movieId, imdbId, tmdbId

The catalogue has about 4,464 movies, the ratings file has about 200,000 ratings, and users have rated about 4.9 movies each on average. Exact counts for movies, ratings, users and genres appear on the How it works page of the app; copy them here for the final submission.

Purpose of EDA

EDA was done before any modelling for three reasons: to confirm the real schema so no column is invented, to find problems that would corrupt joins or training (missing values, duplicates, orphan IDs), and to measure how sparse the ratings are, since that decides what each algorithm can realistically achieve.

Preparation Steps Actually Performed

These steps are implemented in `src/utils.py`.

1. Read all three files and inspected column names, data types and row counts.
2. Checked missing values and duplicate records, and handled both.
3. Converted data types and validated `movieId` and `userId`.
4. Joined the files on `movieId`, keeping the links between them.
5. Split the genre string into individual genres and parsed the release year from the title, which gives the decade used as a content feature.
6. Computed each movie's rating count and average, and a rating-confidence score that weights the average by the count, so a 5.0 from two users does not outrank a 4.6 from thousands.

EDA Exploration - Implementation

The checks below reproduce the exploration with pandas. Adjust the file names to match the files in `data/`, run the script, and paste its output and any plots into this section.

```
import pandas as pd

movies = pd.read_csv("data/movies.csv")      # adjust file names
ratings = pd.read_csv("data/ratings.csv")
links = pd.read_csv("data/links.csv")

for name, df in [("movies", movies), ("ratings", ratings), ("links", links)]:
    print(name, df.shape)
    print(df.dtypes)
    print("missing:\n", df.isna().sum())
    print("duplicate rows:", df.duplicated().sum())

# relationships between the files
print("rated movies not in catalogue:",
      (~ratings["movieId"].isin(movies["movieId"])).sum())
print("catalogue movies never rated:",
      (~movies["movieId"].isin(ratings["movieId"])).sum())

# sparsity and rating behaviour
per_user = ratings.groupby("userId").size()
```

```
per_movie = ratings.groupby("movieId").size()
print("ratings per user (mean, median):", per_user.mean(), per_user.median())
print("ratings per movie (mean, median):", per_movie.mean(), per_movie.median())
print("rating distribution:\n", ratings["rating"].value_counts().sort_index())

# genres
genre_counts = movies["genres"].str.split("|").explode().value_counts()
print(genre_counts)
```

Summary of EDA Findings

Each finding below came from inspecting the actual files, and each changed a design decision.

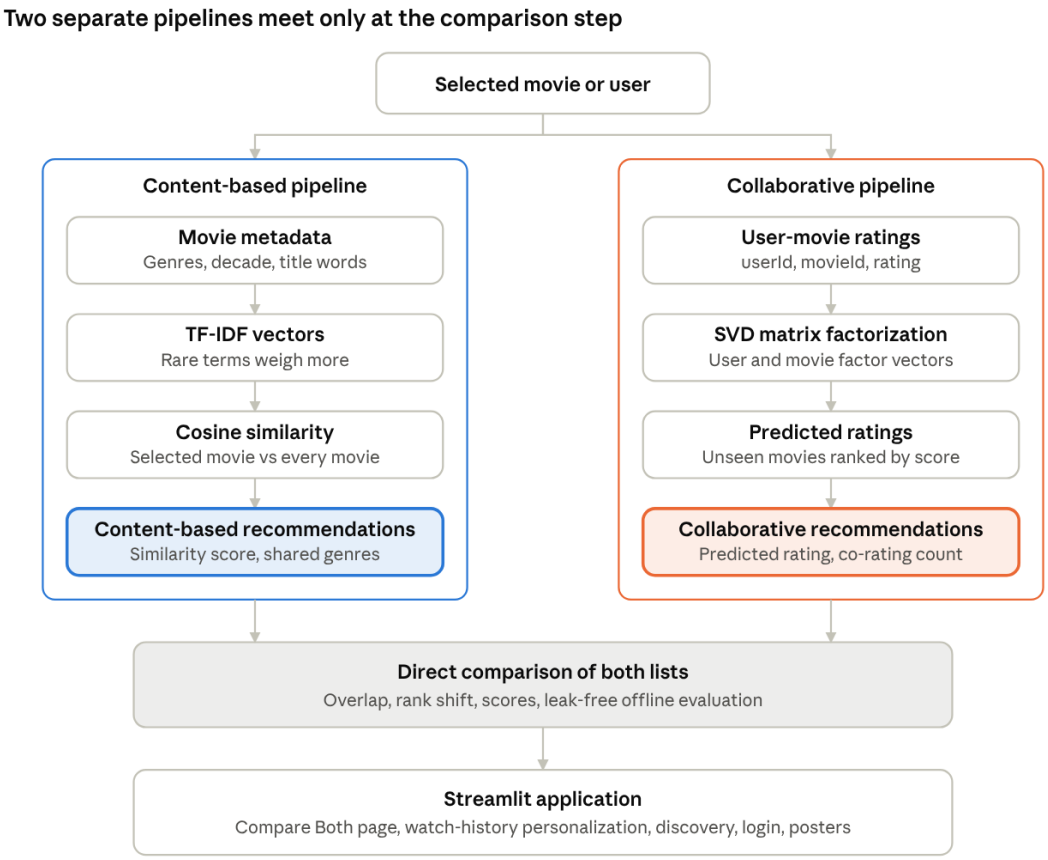
Finding	Effect on the design
Ratings have no timestamp column	No time-based features or claims
The movies file already contains tmdbId	Posters are looked up by TMDB id first, then validated
No keywords, cast or plot in the CSVs	Content features limited to genres, decade and title words
No language or country column	World discovery depends on TMDB metadata only
About 4.9 ratings per user	SVD factors are noisy; hit rates are small; ratings-confidence weighting is needed
About 4,464 movies	Content similarity can be computed over the full catalogue

Algorithm Selection and Implementation

The project needs one method that uses only item metadata and one that uses only ratings, so the two can be compared without sharing a signal. TF-IDF with cosine similarity was chosen for the metadata side because the CSVs hold only genres and titles, and the method is cheap, explainable and needs no ratings. SVD matrix factorization was chosen for the ratings side because the user-movie matrix is very sparse, and factorization learns a short vector per user and per movie instead of requiring direct overlap between users.

Surprise's SVD is used when it is installed, with an automatic NumPy fallback. A rating-confidence score and a popularity list are used only as supporting signals and reference baselines. They are never one of the two compared methods.

System architecture



Architecture · two pipelines, one comparison step

The selected movie or user goes to both pipelines. Each produces its own recommendation list from its own data, and the lists meet only in the comparison step, which feeds the application.

Module	What it does
app.py	Draws every page, button and movie card, the login screen, and keeps session state
src/utils.py	Loads, cleans and joins the CSVs; derives year, genres and the rating-confidence score
src/content_based.py	Content-based pipeline: TF-IDF and cosine similarity

Module	What it does
src/collaborative.py	Collaborative pipeline: SVD, predicted ratings and RMSE
src/compare.py	Runs both methods on the same movie or user; overlap, rank shift and offline evaluation
src/recommender.py	Mixes the signals for the home rows, mood, Surprise Me, Hidden Gems, Movie Night and Double Feature
src/personalization.py	Taste bars, movie profile, achievements and the mood-to-genre lists
src/poster_utils.py	TMDB lookup, title and year validation, age-rating filter, scene and generated fallbacks
src/auth.py	Sign-up and login with hashed passwords; saves watch history and My List
src/theme.py	Colours, glow, hover effects and card styling
tests/test_engine.py	Automated tests for the engine, login and comparison

Content-based filtering

Content-based filtering recommends movies whose metadata resembles the selected movie. It never looks at other users' ratings.

The code is in src/content_based.py (build_content_model() and get_content_recommendations()).

Pipeline

1. Combine each movie's genres, decade and title words into one text document.
2. Convert the documents to TF-IDF vectors, which give more weight to terms that are rare across the catalogue.
3. Compute cosine similarity between the selected movie's vector and every other movie's vector.
4. Sort by similarity, remove the selected movie and duplicates, and return the top results.

Cosine similarity between two movie vectors A and B:

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|}$$

Explanation shown to the user: the shared genres and the cosine score. Unknown titles are handled safely and return no recommendations instead of an error.

Example: selecting Toy Story (1995) gives the "Because you watched Toy Story" row, filled with animated and family movies that share its genres.

Strengths: works for any movie, including ones with few ratings; easy to explain. **Weakness:** limited to genres, decade and title words, so it cannot tell two movies in the same genre apart by quality or by who liked them.

Collaborative filtering

Collaborative filtering learns from rating patterns: users who rated movies similarly are assumed to like similar new movies. It uses only userId, movieId and rating, and none of the genre or title text. The code is in `src/collaborative.py` (`build_svd_model()` and `get_svd_recommendations()`).

Method: - SVD matrix factorization represents each user and each movie as a short vector of hidden factors. The predicted rating for user u and movie i is:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^{\top} p_u$$

Here μ is the global mean rating, b_u and b_i are user and movie biases, and p_u and q_i are the user and movie factor vectors. Unseen movies are ranked by predicted rating.

Settings and result

- Hyperparameters chosen by a regularization sweep: 20 factors, regularization 0.15.
- Training takes about 4 seconds, and the model is cached so it is not retrained on each click.
- Hold-out RMSE is about 0.898, against 0.9025 for a bias-only baseline.
- The app uses Surprise's SVD when it is installed. If Surprise cannot be installed, it falls back automatically to a NumPy SVD. All tests in development used the fallback.

Explanation shown to the user: the number of users who rated both movies, and in user mode the SVD predicted rating. The page states that the recommendations come from learned rating patterns.

Strengths: captures taste patterns that genres cannot, such as two movies from different genres liked by the same people.

Weakness: with about 4.9 ratings per user, the learned factors are noisy, and SVD neighbours of a single movie performed close to chance. SVD works better for predicting ratings than for finding look-alike movies.

Comparison methodology and results

The two methods recommend almost completely different movies, and SVD is slightly better at predicting ratings. The comparison is in `src/compare.py` and shown on the Compare Both page.

Single-movie and single-user comparison. For a selected movie (or a real MovieLens user), the page runs both pipelines and reports:

- movies recommended by both methods, only by content-based, and only by collaborative filtering;
- the rank of each movie in each list, and the rank shift between them;
- overlap as a Jaccard score, and rank correlation;
- bars for genre match, popularity and cross-genre picks, and a Venn-style overlap diagram.

Offline evaluation: -

The evaluation is leak-free: each method is trained only on a user's training ratings, with one movie that the user rated 4 or higher hidden. Each method then produces 10 recommendations, and a hit means the hidden movie appears. This covered 4,682 users. An earlier version let the SVD see the hidden movie, so its numbers were discarded.

Rating prediction error (RMSE, lower is better)

Method	RMSE
Item-mean baseline	0.961
Content-based	0.909

Method	RMSE
SVD	0.898

Finding a hidden liked movie (top 10)

Method	Hit rate	Note
Content-based	1.15%	Similarity to the user's liked movies
SVD neighbours of one movie	1.28%	Close to chance
SVD predicted ratings	1.73%	Best of the two real methods
Popularity list	3.4%	Recommends only about 0.3% of the catalogue

Interpretation

- The top-10 lists of the two methods overlap by about 0.2%. They use different information, so they surface different movies, which is the reason to compare them.
- A popularity list scores the highest hit rate but gives nearly every user the same few movies. It is useless for discovery, so it is shown only as a reference.
- All hit rates are small because the data is sparse (about 4.9 ratings per user across about 4,464 movies). Read them as relative comparisons, not as absolute quality.
- Content-based filtering remains the better signal for "similar movies" rows, where the goal is similarity and not guessing a hidden rating.

Application layer and features

The interface has a login page and five main pages. Everything beyond the Compare Both page is built on the same recommendation signals.

Page	What it offers
Login / sign-up	Poster-collage landing page, log in, sign up, or continue as guest. Accounts are local, with hashed

Page	What it offers
	passwords. A signed-in user's watch history, My List, moods and preferences are saved and restored.
Home	Hero banner, Today's Pick, Trending for You, mood picker, "Because you watched", "Made for your watch history", and a collaborative filtering row.
Compare Both	The core of the project: content-based and collaborative recommendations side by side for a chosen movie or MovieLens user, with overlap, rank shift and evaluation.
Discover	Surprise Me, Pick One (movie face-off), Movie Night, Double Feature, One Movie Before Bed, Hidden Gems.
Explore	Browse by genre, only genres that exist in the dataset.
My Space	My List, watch history, taste bars, movie profile and achievements.
How it works	Plain explanation of each method, dataset statistics and RMSE.

Personalization: -

Watch history (most recent first, deduplicated, capped at 20) feeds a genre profile. The profile drives "Because you watched", the combined "Made for your watch history" row, the taste bars, and the movie profile label. The label is derived only from the actual genre distribution.

Mood and discovery: -

The user picks a viewing mood, which is mapped to genres and combined with the history profile. Discovery mode (Familiar, Explore, Wild) is a diversity control that changes how far results move from the history profile. Neither claims to read emotions or personality.

Other behaviour: -

Hidden Gems combines rating quality, rating count and content relevance. Recommendations normally exclude watched movies, My List movies and recently shown movies. Achievements (first movie, 5 and 10 watched, 5 genres, 5 moods, 5 saved) unlock only from real actions.

Artwork: -

A movie card shows the TMDB poster. If no poster exists, a scene image is used. If neither exists, a generated cinematic card is drawn from the title, genre and year, so no card is ever blank. A strict family-friendly filter (the default) shows real artwork only when TMDB confirms a G, PG or PG-13 style rating.

Implementation details

Technology stack. Python, Streamlit, Pandas, NumPy, scikit-learn, Surprise (SVD), Requests and Pillow. The TMDB API supplies posters and metadata only. It is never used to generate recommendations.

TMDB artwork pipeline: -

For each movie the app tries, in order:

1. the TMDB movie found by the MovieLens `tmdbId`, with its title and year checked;
2. an alternate poster from TMDB's poster list, English first;
3. a scene image (backdrop);
4. a TMDB title search, validated by title and year and never by blindly taking the first result;
5. a generated cinematic card built from the title, genre and year.

So, no card is ever blank. Requests retry once on timeouts and rate limits, and a circuit breaker stops repeated calls during an outage. If `api.themoviedb.org` is unreachable, the app switches to the alternate address `api.tmdb.org`.

Age-rating filter: -

The default Strict level shows a real poster or scene only when TMDB confirms G, PG or PG-13 (or the Indian U and UA, and British U and PG, equivalents). R, NC-17, Indian A, UK 18, adult-flagged titles and

titles with no rating data get a generated card tagged as age-rated. Standard hides only R and 18+ content. Off shows everything.

Performance: -

The TF-IDF model, similarity matrix and SVD model are built once and cached with `st.cache_resource`. Processed data and TMDB responses are cached with `st.cache_data`, so a button click never retrains a model or repeats a poster request.

State and accounts: -

Watch history, My List, selected mood, face-off picks, recent recommendations and Surprise Me history live in Streamlit session state, with unique widget keys. Signed-in users have this state saved to a local file and restored at the next login. Passwords are stored hashed.

Error handling: -

Missing TMDB id, poster or backdrop, API failure, empty history, empty list, unknown search, missing rating, duplicate records and unavailable SVD are all handled without crashing the page.

Testing and validation

Testing covered the engine and the interface, but not a real browser session.

Test	Result
Engine tests (joins, content similarity, SVD, multi-history, moods, hidden gems, Surprise Me with no repeats, deterministic Today's Pick, comparison, login)	16 tests pass
Interface scenarios with a stubbed Streamlit and a mocked TMDB (watch history, My List, Compare Both, login and sign-up, artwork rules)	No crashes, no duplicate widget keys, no malformed HTML
SVD hold-out RMSE	About 0.898 against a 0.9025 bias-only baseline
Artwork rules against a simulated TMDB (poster, alternate poster, scene, generated, each age-filter level)	Behaved as specified

Not verified during development: the app was not run in real Streamlit, so layout and CSS were not seen on screen. Real TMDB calls were not made from the build environment, and Surprise SVD was not installed there. Before presenting, run the app, press Test connection in the Posters panel, and check that `scikit-surprise` installed.

Installation and running

1. Unzip the project into a new, empty folder.
2. Run `pip install -r requirements.txt`. If `scikit-surprise` fails to install, remove that line. The app then uses its built-in NumPy SVD.
3. Add the TMDB key to `.streamlit/secrets.toml` as `TMDB_API_KEY = "your_key"`, or set the `TMDB_API_KEY` environment variable.
4. Run `streamlit run app.py`.
5. Open the Posters panel and press Test connection. If TMDB cannot be reached, try another network or a different DNS (1.1.1.1 or 8.8.8.8). Cards then show generated artwork until the connection works.

Conclusion

Content-based and collaborative filtering recommend almost entirely different movies from the same data: the overlap of their top-10 lists was about 0.2%. They answer different questions. Content-based filtering finds movies that resemble a chosen movie, using only genres, decade and title words. Collaborative filtering predicts how a user would rate unseen movies, using only ratings.

On a leak-free evaluation over 4,682 users, SVD predicted ratings best (RMSE about 0.898, against about 0.909 for content-based scoring and about 0.961 for an item-mean baseline) and found a hidden liked movie more often (1.73% against 1.15%). SVD was close to chance at movie-to-movie similarity, so content-based filtering remains the better choice for "movies like this one". A popularity list scored higher on hits (3.4%) but covers only about 0.3% of the catalogue. The sparse ratings, about 4.9 per user, limit every number reported, so the results should be read as relative comparisons on this dataset.

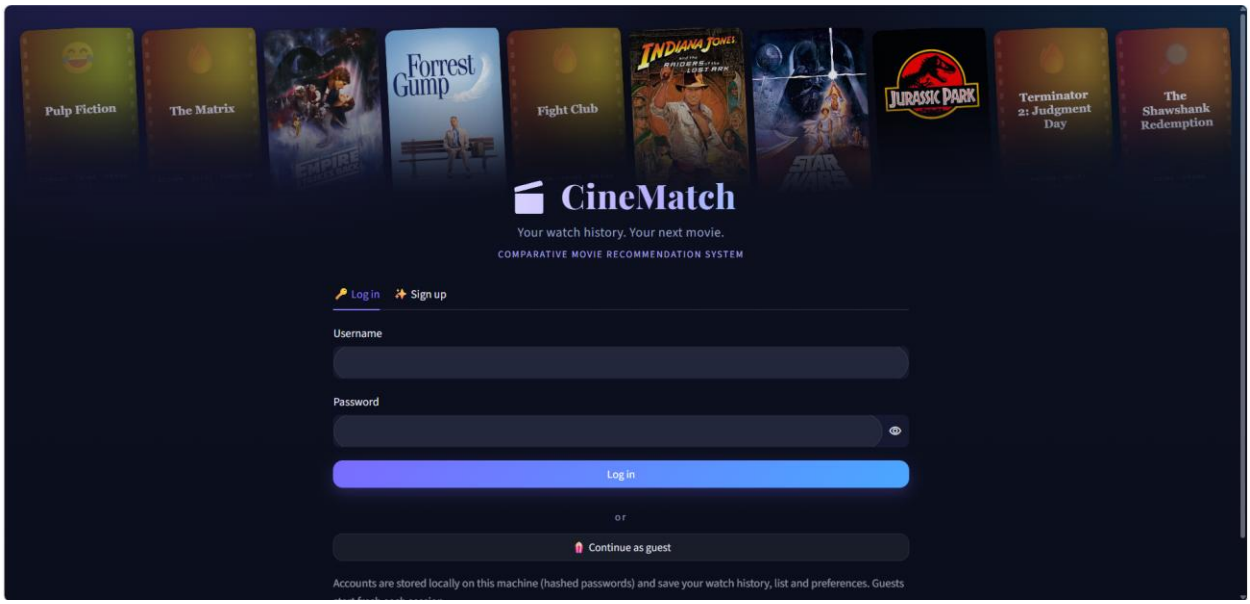
CineMatch delivers this comparison inside a complete application, with a Compare Both page that shows the two pipelines' outputs side by side, and a personalized, login-based interface built on the same signals.

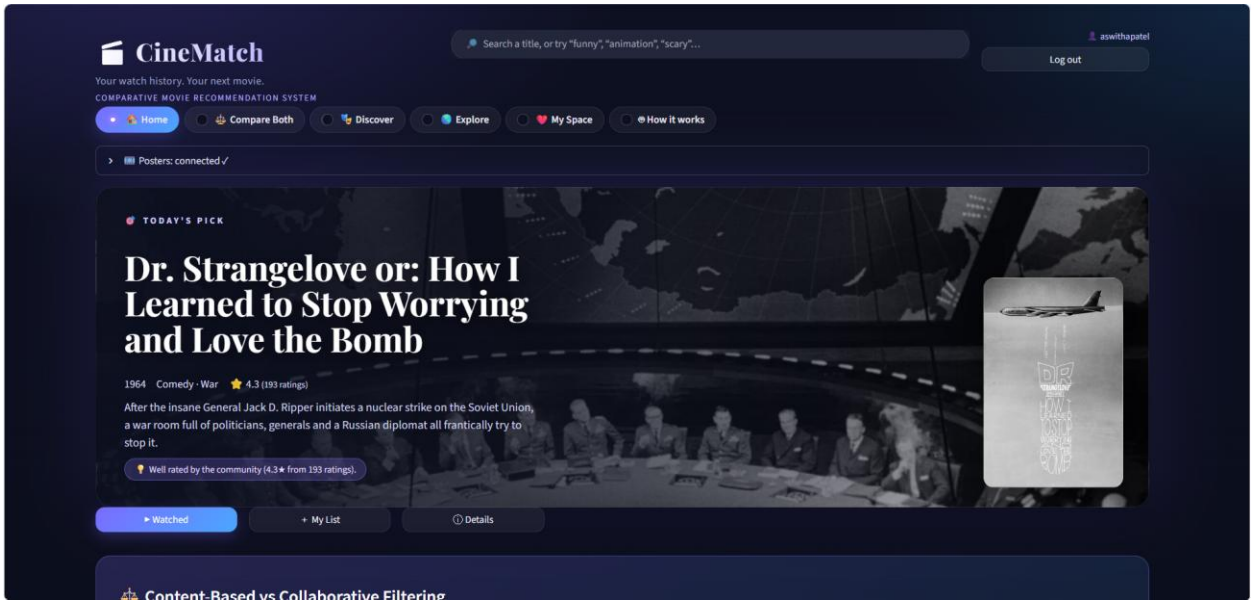
Future work

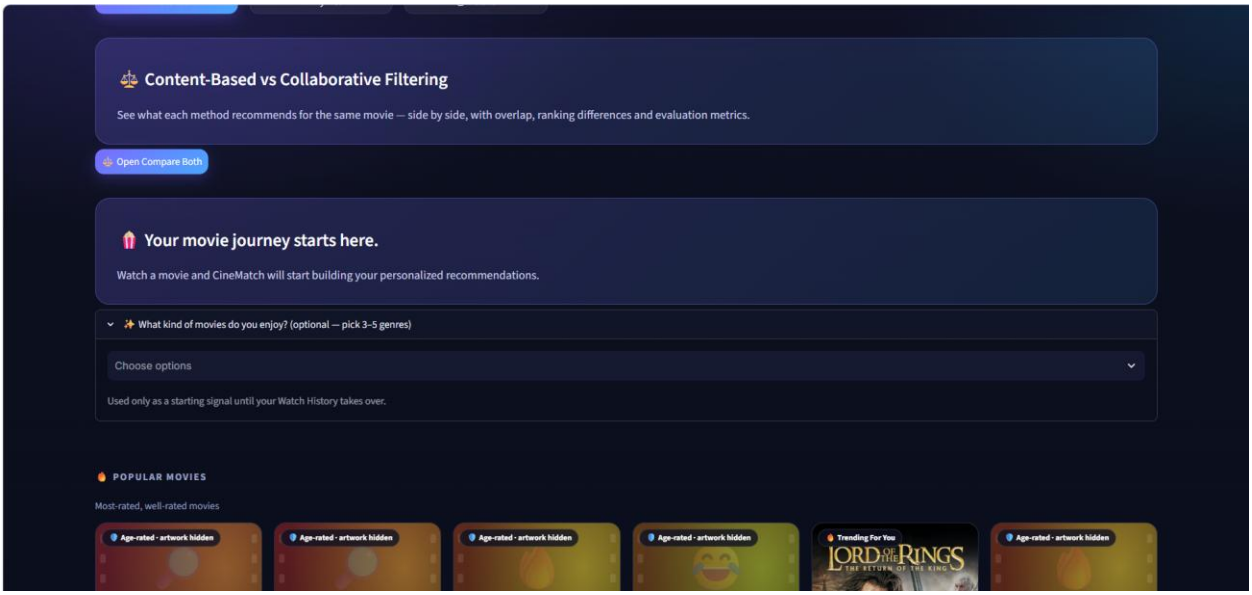
- Add richer content features, such as keywords, cast and plot overviews, which the CSVs lack.
- Combine the two methods in a tuned hybrid and evaluate it against the same leak-free protocol.

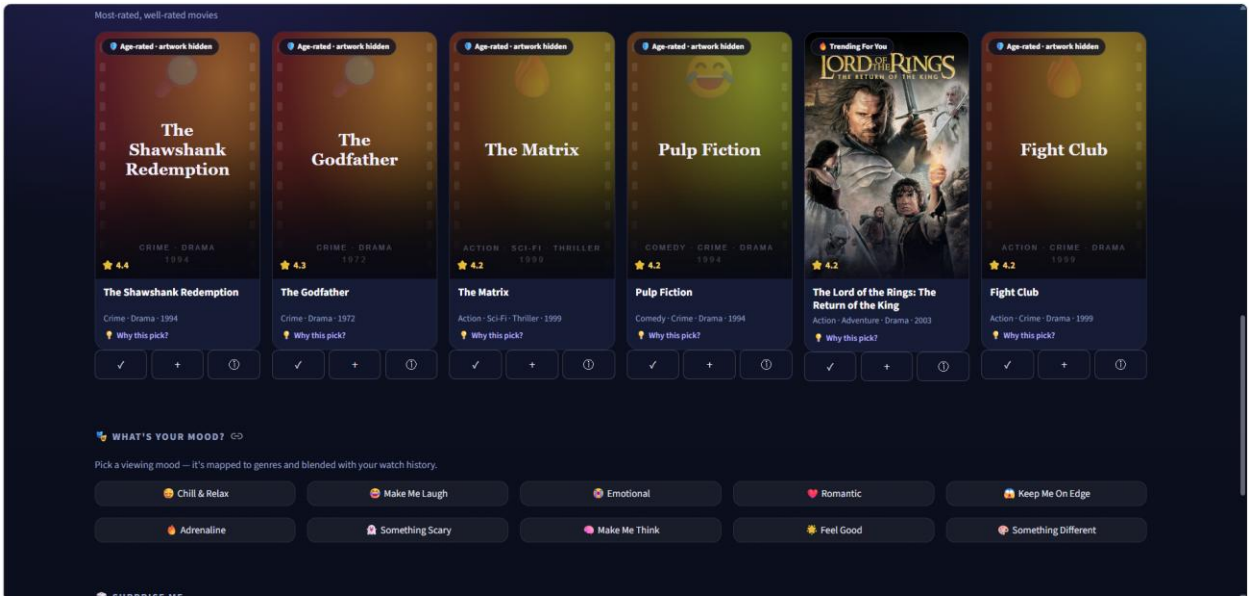
- Evaluate on a denser ratings dataset, with timestamps, and with more than one random seed.
- Test the interface with real viewers and add online measures such as click-through.
- Replace the local login with a proper authentication service.

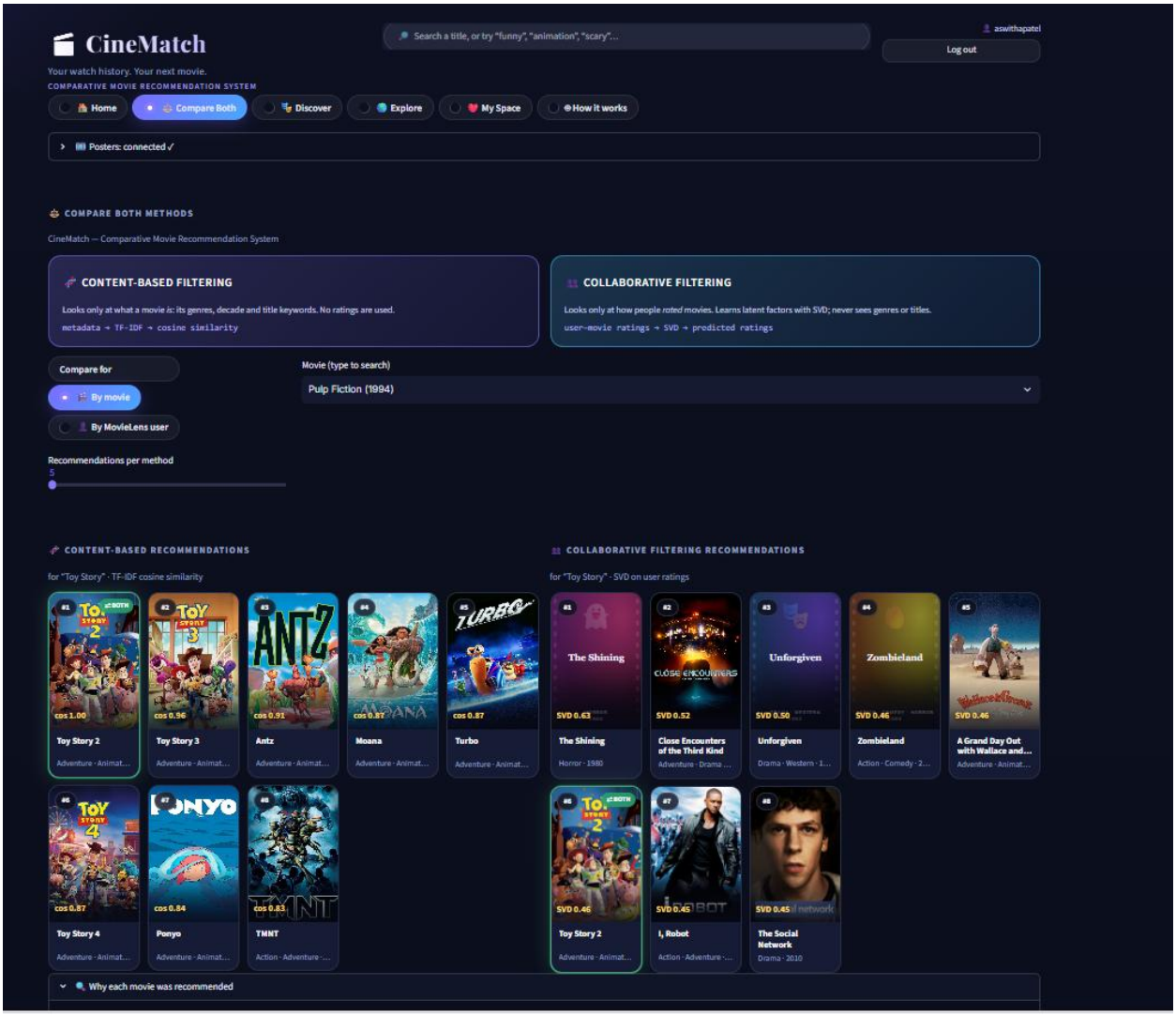
Webpage Screenshots

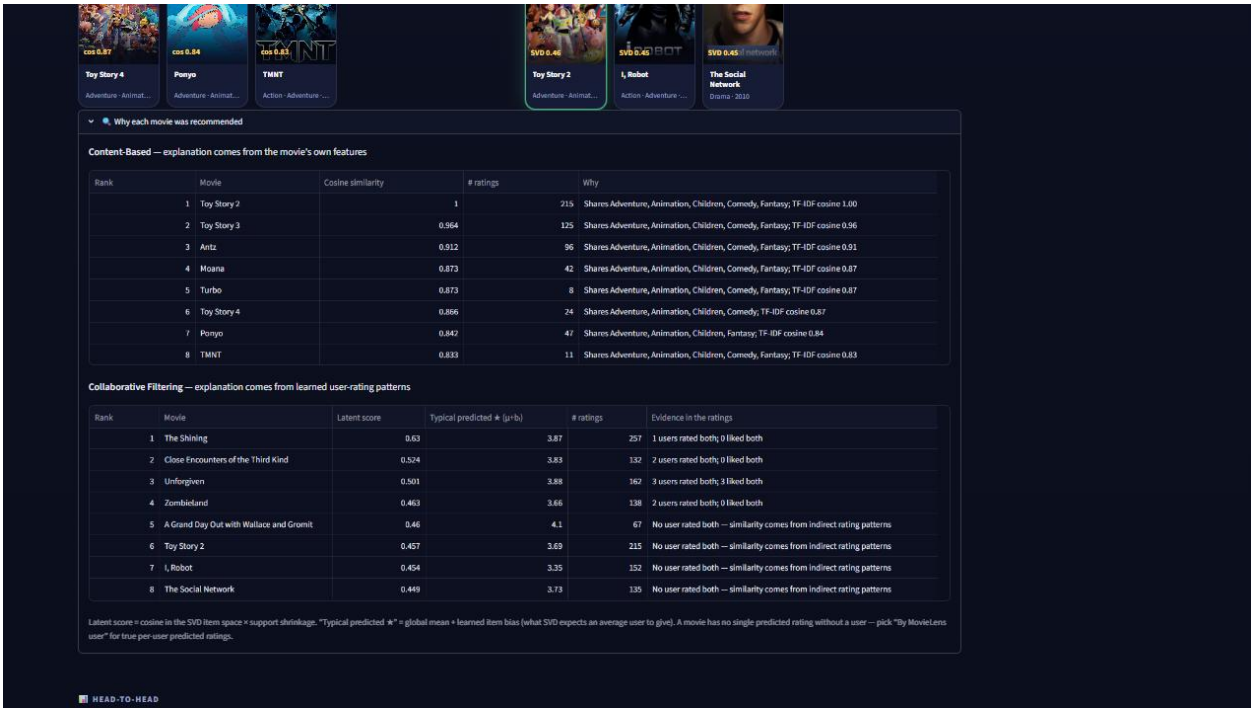


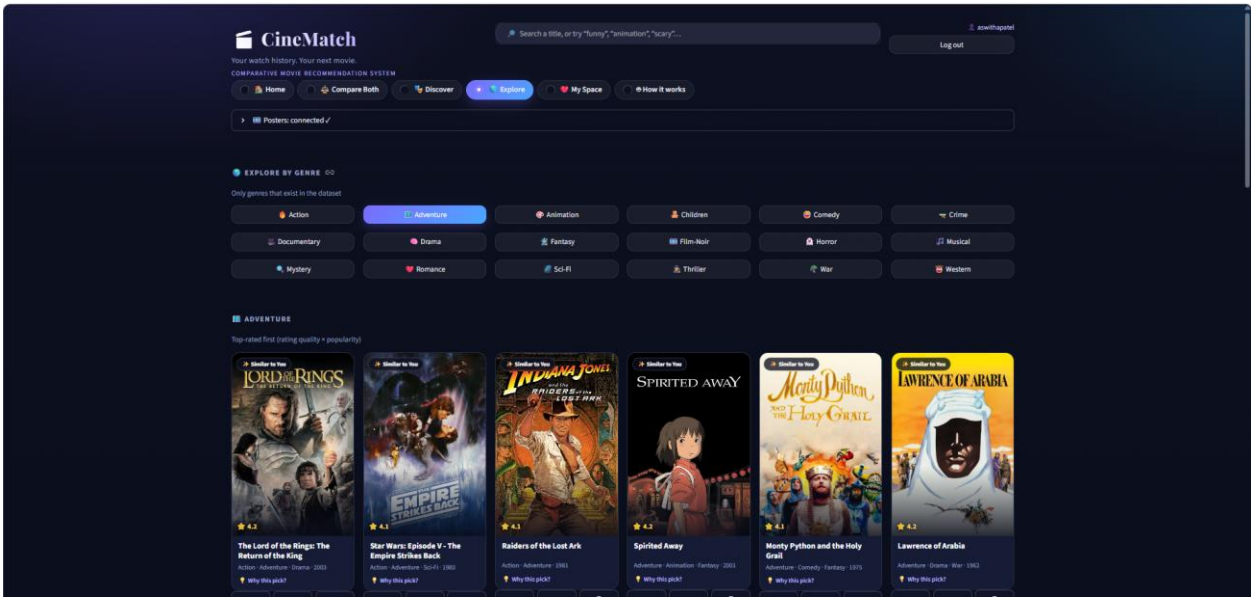


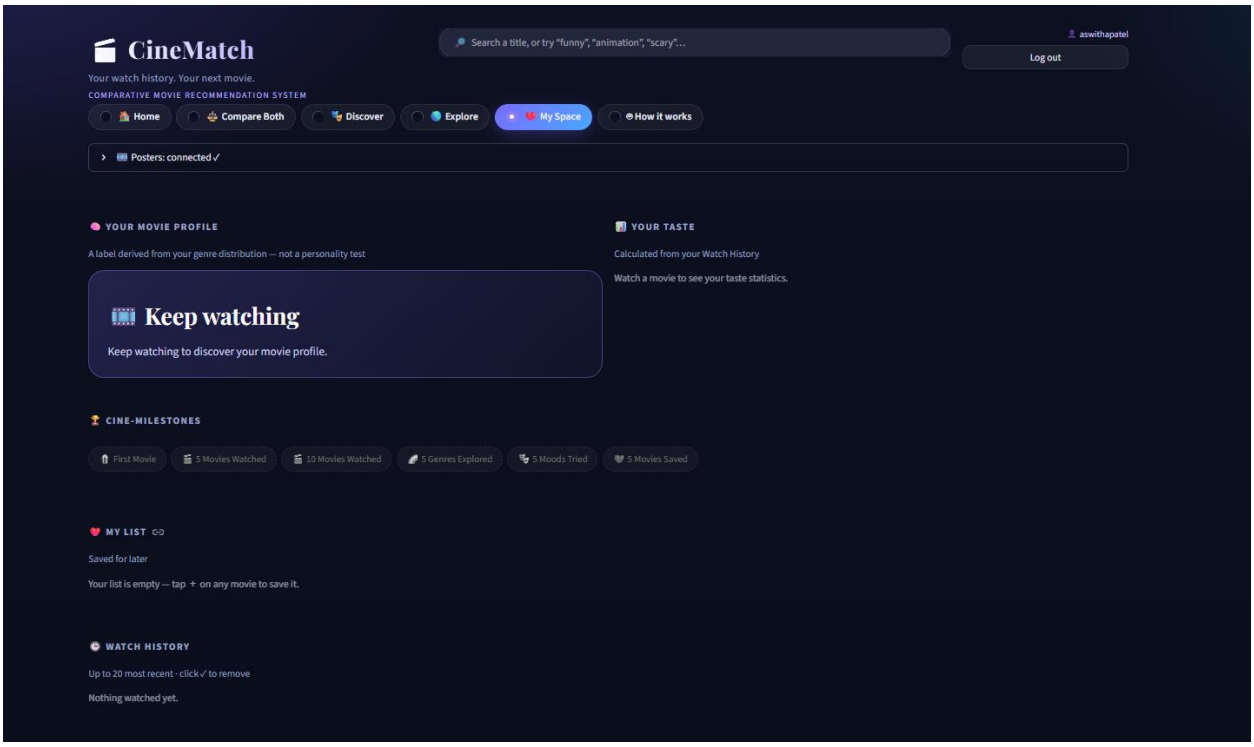

Content-Based vs Collaborative Filtering
See what each method recommends for the same movie — side by side, with overlap, ranking differences and evaluation metrics.
Open Compare Both
Your movie journey starts here.
Watch a movie and CineMatch will start building your personalized recommendations.
What kind of movies do you enjoy? (optional — pick 3–5 genres)
Choose options
Used only as a starting signal until your Watch History takes over.
POPULAR MOVIES
Most-rated, well-rated movies
Age-rated · artwork hidden
Trending For You
LORD OF THE RINGS
THE RETURN OF THE KING


Most-rated, well-rated movies
Age-rated · artwork hidden
The Shawshank Redemption
CRIME · DRAMA
1994
4.4
The Shawshank Redemption
Crime · Drama · 1994
Why this pick?
The Godfather
CRIME · DRAMA
1972
4.3
The Godfather
Crime · Drama · 1972
Why this pick?
The Matrix
ACTION · SCI-FI · THRILLER
1999
4.2
The Matrix
Action · Sci-Fi · Thriller · 1999
Why this pick?
Pulp Fiction
COMEDY · CRIME · DRAMA
1994
4.2
Pulp Fiction
Comedy · Crime · Drama · 1994
Why this pick?
Trending For You
LORD OF THE RINGS
THE RETURN OF THE KING
4.2
The Lord of the Rings: The Return of the King
Action · Adventure · Drama · 2003
Why this pick?
Fight Club
ACTION · CRIME · DRAMA
1999
4.2
Fight Club
Action · Crime · Drama · 1999
Why this pick?
WHAT'S YOUR MOOD?
Pick a viewing mood — it's mapped to genres and blended with your watch history.
Chill & Relax
Make Me Laugh
Emotional
Romantic
Keep Me On Edge
Adrenaline
Something Scary
Make Me Think
Feel Good
Something Different


CineMatch
Search a title, or try "funny", "animation", "scary"...
aswithapatel
Log out
Your watch history. Your next movie.
COMPARATIVE MOVIE RECOMMENDATION SYSTEM
Home
Compare Both
Discover
Explore
My Space
How it works
Posters: connected ✓
COMPARE BOTH METHODS
CineMatch — Comparative Movie Recommendation System
CONTENT-BASED FILTERING
Looks only at what a movie is: its genres, decade and title keywords. No ratings are used.
metadata → TF-IDF → cosine similarity
COLLABORATIVE FILTERING
Looks only at how people rated movies. Learns latent factors with SVD; never sees genres or titles.
user–movie ratings → SVD → predicted ratings
Compare for
By movie
By MovieLens user
Movie (type to search)
Pulp Fiction (1994)
Recommendations per method
5
CONTENT-BASED RECOMMENDATIONS
for "Toy Story" · TF-IDF cosine similarity
cos 1.00
Toy Story 2
Adventure · Animat...
cos 0.96
Toy Story 3
Adventure · Animat...
cos 0.91
Antz
Adventure · Animat...
cos 0.87
Moana
Adventure · Animat...
cos 0.87
Turbo
Adventure · Animat...
COLLABORATIVE FILTERING RECOMMENDATIONS
for "Toy Story" · SVD on user ratings
SVD 0.63
The Shining
Horror · 1980
SVD 0.52
Close Encounters of the Third Kind
Adventure · Drama ...
SVD 0.50
Unforgiven
Drama · Western · 1...
SVD 0.46
Zombieland
Action · Comedy · 2...
SVD 0.46
A Grand Day Out with Wallace and...
Adventure · Animat...
cos 0.87
Toy Story 4
Adventure · Animat...
cos 0.84
Ponyo
Adventure · Animat...
cos 0.83
TMNT
Action · Adventure · ...
SVD 0.46
Toy Story 2
Adventure · Animat...
SVD 0.45
I, Robot
Action · Adventure · ...
SVD 0.45
The Social Network
Drama · 2010
Why each movie was recommended


cos 0.87
Toy Story 4
Adventure · Animat…
cos 0.84
Ponyo
Adventure · Animat…
cos 0.83
TMNT
Action · Adventure ·…
SVD 0.46
Toy Story 2
Adventure · Animat…
SVD 0.45
I, Robot
Action · Adventure ·…
SVD 0.45
The Social Network
Drama · 2010
Why each movie was recommended
Content-Based — explanation comes from the movie's own features
Rank
Movie
Cosine similarity
ratings
Why
1 Toy Story 2 1 215 Shares Adventure, Animation, Children, Comedy, Fantasy; TF-IDF cosine 1.00
2 Toy Story 3 0.964 125 Shares Adventure, Animation, Children, Comedy, Fantasy; TF-IDF cosine 0.96
3 Antz 0.912 96 Shares Adventure, Animation, Children, Comedy, Fantasy; TF-IDF cosine 0.91
4 Moana 0.873 42 Shares Adventure, Animation, Children, Comedy, Fantasy; TF-IDF cosine 0.87
5 Turbo 0.873 8 Shares Adventure, Animation, Children, Comedy, Fantasy; TF-IDF cosine 0.87
6 Toy Story 4 0.866 24 Shares Adventure, Animation, Children, Comedy; TF-IDF cosine 0.87
7 Ponyo 0.842 47 Shares Adventure, Animation, Children, Fantasy; TF-IDF cosine 0.84
8 TMNT 0.833 11 Shares Adventure, Animation, Children, Comedy, Fantasy; TF-IDF cosine 0.83
Collaborative Filtering — explanation comes from learned user-rating patterns
Rank
Movie
Latent score
Typical predicted ★ (μ+b)
ratings
Evidence in the ratings
1 The Shining 0.63 3.87 257 1 users rated both; 0 liked both
2 Close Encounters of the Third Kind 0.524 3.83 132 2 users rated both; 0 liked both
3 Unforgiven 0.501 3.88 162 3 users rated both; 3 liked both
4 Zombieland 0.463 3.66 138 2 users rated both; 0 liked both
5 A Grand Day Out with Wallace and Gromit 0.46 4.1 67 No user rated both — similarity comes from indirect rating patterns
6 Toy Story 2 0.457 3.69 215 No user rated both — similarity comes from indirect rating patterns
7 I, Robot 0.454 3.35 152 No user rated both — similarity comes from indirect rating patterns
8 The Social Network 0.449 3.73 135 No user rated both — similarity comes from indirect rating patterns
Latent score = cosine in the SVD item space × support shrinkage. "Typical predicted ★" = global mean + learned item bias (what SVD expects an average user to give). A movie has no single predicted rating without a user — pick "By MovieLens user" for true per-user predicted ratings.
HEAD-TO-HEAD


CineMatch
Your watch history. Your next movie.
COMPARATIVE MOVIE RECOMMENDATION SYSTEM
Search a title, or try "funny", "animation", "scary"…
aswithapatel
Log out
Home
Compare Both
Discover
Explore
My Space
How it works
Posters: connected ✓
EXPLORE BY GENRE
Only genres that exist in the dataset
Action
Adventure
Animation
Children
Comedy
Crime
Documentary
Drama
Fantasy
Film-Noir
Horror
Musical
Mystery
Romance
Sci-Fi
Thriller
War
Western
ADVENTURE
Top-rated first (rating quality × popularity)
Similar to You
The Lord of the Rings: The Return of the King
Action · Adventure · Drama · 2003
Why this pick?
Star Wars: Episode V - The Empire Strikes Back
Action · Adventure · Sci-Fi · 1980
Raiders of the Lost Ark
Action · Adventure · 1981
Spirited Away
Adventure · Animation · Fantasy · 2001
Monty Python and the Holy Grail
Adventure · Comedy · Fantasy · 1975
Lawrence of Arabia
Adventure · Drama · War · 1962


CineMatch
Your watch history. Your next movie.
COMPARATIVE MOVIE RECOMMENDATION SYSTEM
Search a title, or try "funny", "animation", "scary"...
aswithapatel
Log out
Home
Compare Both
Discover
Explore
My Space
How it works
Posters: connected ✓
YOUR MOVIE PROFILE
A label derived from your genre distribution — not a personality test
Keep watching
Keep watching to discover your movie profile.
YOUR TASTE
Calculated from your Watch History
Watch a movie to see your taste statistics.
CINE-MILESTONES
First Movie
5 Movies Watched
10 Movies Watched
5 Genres Explored
5 Moods Tried
5 Movies Saved
MY LIST
Saved for later
Your list is empty — tap + on any movie to save it.
WATCH HISTORY
Up to 20 most recent · click ✓ to remove
Nothing watched yet.